Mensajería con colas
y RabbitMQ en Pedidos360

Conceptos, arquitecturas y patrones de la mensajería asíncrona; cómo programarla con Spring AMQP; y explicación detallada de toda la configuración RabbitMQ del proyecto, caso por caso.

DSY1107 · Desarrollo Cloud Native I · Evaluación Parcial N° 3
Octubre de 2026

Contenido

1. Qué es la mensajería con colas . . . 4

1.1 Ideas clave . . . 4

2. Para qué sirve: beneficios y costos . . . 5

2.1 Beneficios . . . 5

2.2 Costos y desafíos . . . 5

3. Cuándo usar mensajería y cuándo no . . . 6

4. RabbitMQ y el modelo AMQP . . . 6

4.1 Componentes . . . 6

4.2 Tipos de exchange . . . 7

5. Garantías de entrega y confirmaciones . . . 8

5.1 Semánticas de entrega . . . 8

5.2 Confirmaciones del consumidor: ACK, NACK y reject . . . 8

5.3 Confirmaciones del publicador . . . 8

6. Manejo de errores: reintentos, DLX y DLQ . . . 9

6.1 Errores recuperables y no recuperables . . . 9

6.2 Dead Letter Exchange (DLX) y Dead Letter Queue (DLQ) . . . 9

6.3 Estrategias de reintento . . . 9

7. Arquitecturas y patrones de mensajería . . . 10

7.1 Patrones de comunicación . . . 10

7.2 Tipos de mensaje: comando, evento y consulta . . . 11

7.3 Patrones de confiabilidad . . . 11

7.4 Coordinación entre servicios: coreografía, orquestación y sagas . . . 11

7.5 Por qué RabbitMQ: comparación con otras alternativas . . . 12

7.6 Tipos de cola en RabbitMQ . . . 12

8. Cómo programarlo con Spring Boot y Spring AMQP . . . 12

8.1 Paso 1: dependencia . . . 12

8.2 Paso 2: conexión y nombres en application.yml . . . 12

8.3 Paso 3: declarar la topología con beans . . . 13

8.4 Paso 4: publicar (productor) . . . 13

8.5 Paso 5: consumir (consumidor) . . . 14

8.6 Paso 6: administración programática . . . 14

8.7 Buenas prácticas . . . 14

9. RabbitMQ en Pedidos360: la topología completa . . . 14

9.1 Inventario de recursos . . . 15

9.2 Parámetros configurables . . . 16

9.3 Infraestructura: el servicio rabbitmq en Docker Compose . . . 16
10. El código, servicio por servicio . . . 17
10.1 Organización de los paquetes . . . 17
10.2 msvc-pago: el productor con confirmaciones . . . 17
10.3 msvc-envio: configuración por caso de uso . . . 19
10.4 La clase base de los consumidores . . . 21
10.5 Caso de uso 1: registro del envío . . . 23
10.6 Caso de uso 2: notificación por correo . . . 24
10.7 Caso de uso 3: msvc-pago como consumidor . . . 25
10.8 Los consumidores de las DLQ . . . 26
10.9 msvc-rabbitmq-admin: la API de administración . . . 27
11. Qué pasa en cada caso . . . 29
Caso 1: compra exitosa . . . 29
Caso 2: correo enviado y aviso a msvc-pago . . . 29
Caso 3: mensajes duplicados . . . 30
Caso 4: evento con datos incompletos (error no recuperable) . . . 30
Caso 5: JSON inválido . . . 30
Caso 6: la base de datos falla (error recuperable) y se agotan los reintentos . . . 31
Caso 7: el aviso de correo llega antes que el pedido . . . 31
Caso 8: msvc-envio está detenido . . . 31
Caso 9: RabbitMQ no confirma la publicación . . . 32
Caso 10: administración por API . . . 32
12. Cómo comprobar que funciona . . . 32
12.1 En local (Docker Desktop) . . . 32
12.2 En AWS (EC2 backend, por Session Manager) . . . 32
12.3 Pruebas automáticas . . . 33
13. Evaluación del diseño y mejoras propuestas . . . 33
13.1 Lo que está bien resuelto . . . 33
13.2 Mejoras pendientes . . . 33
14. Resumen: por qué Pedidos360 necesita una cola de mensajes . . . 34
15. Glosario . . . 34
Referencias . . . 35

1. Qué es la mensajería con colas

En una arquitectura de microservicios los servicios necesitan hablar entre sí. Hay dos grandes formas de hacerlo. En la **comunicación síncrona** (por ejemplo, una llamada HTTP/REST) el servicio que llama envía una petición y **queda esperando** la respuesta: si el otro servicio está lento o caído, quien llama también se ve afectado. En la **comunicación asíncrona por mensajes**, el servicio deja un mensaje en un intermediario (el **broker**) y sigue con su trabajo; otro servicio retira ese mensaje cuando puede y lo procesa.

Una **cola de mensajes** es justamente ese buzón intermedio: una estructura que guarda mensajes en orden de llegada (FIFO) hasta que un consumidor los retira y confirma que los procesó. El broker se encarga de almacenarlos de forma segura, entregarlos, reintentarlos y, si nadie puede procesarlos, apartarlos.

Una analogía útil es la diferencia entre una **llamada telefónica** y un **correo postal**. En la llamada ambos deben estar disponibles al mismo tiempo; si el otro no contesta, la comunicación falla. En el correo, el remitente deja la carta en el buzón y se olvida: la oficina de correos la guarda y la entrega cuando el destinatario está disponible.

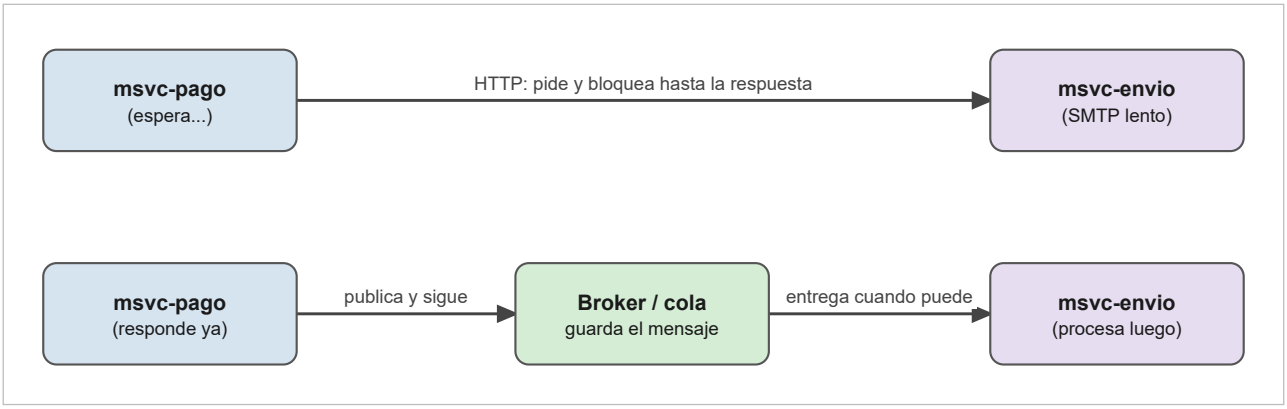


Figura 1. Arriba, comunicación síncrona: quien llama depende del otro. Abajo, asíncrona: el broker desacopla a ambos.

1.1 Ideas clave

- **Productor (publisher)**: el servicio que crea y envía el mensaje. No sabe quién lo va a procesar.
- **Broker**: el servidor de mensajería (RabbitMQ, Kafka, Amazon SQS…). Recibe, guarda y entrega.
- **Cola (queue)**: el buffer donde esperan los mensajes. Persiste aunque no haya consumidores conectados.
- **Consumidor (consumer)**: el servicio que retira mensajes y los procesa.
- **Confirmación (ACK)**: el aviso del consumidor al broker de que el mensaje se procesó y puede borrarse. Sin ACK, el broker considera que el mensaje sigue pendiente.
- **Mensaje**: un paquete con *cuerpo* (los datos, normalmente JSON) y *propiedades/headers* (metadatos: tipo de contenido, identificadores, número de reintento, etc.).

2. Para qué sirve: beneficios y costos

La mensajería no es un fin en sí misma; resuelve problemas concretos de los sistemas distribuidos. Conviene conocer tanto lo que aporta como lo que cuesta.

2.1 Beneficios

Beneficio	Qué significa	Ejemplo en Pedidos360
Desacoplamiento	El productor no conoce al consumidor: ni su dirección, ni cuántos hay, ni si están vivos. Se pueden agregar consumidores nuevos sin tocar al productor.	msvc-pago publica *pago.completado* y no sabe que msvc-envio existe.
Asincronía y latencia	La operación principal responde de inmediato; el trabajo secundario ocurre después.	El checkout responde sin esperar a que se registre el envío ni a que se mande el correo.
Resiliencia	Si el consumidor está caído, los mensajes esperan en la cola y se procesan cuando vuelve. Una falla no se propaga en cadena.	Si msvc-envio o su base de datos fallan, la compra igual se confirma; el envío se procesa después.
Nivelación de carga	La cola absorbe picos: el consumidor procesa a su ritmo (prefetch) sin colapsar.	En un Cyber Day, 1.000 pagos por minuto se encolan y msvc-envio los procesa a su velocidad.
Escalabilidad horizontal	Varias instancias del mismo consumidor compiten por la cola y se reparten el trabajo.	Levantar 3 réplicas de msvc-envio triplica la capacidad sin cambiar código.
Manejo de errores controlado	Reintentos, colas de mensajes muertos (DLQ) y trazabilidad de fallos.	Los errores temporales se reintentan; los irrecuperables van a *envio.dlq* y quedan en logs.

2.2 Costos y desafíos

La mensajería agrega complejidad que hay que gestionar conscientemente:

- **Consistencia eventual**: durante unos instantes el pedido existe en msvc-pago pero el envío aún no existe en msvc-envio. El sistema queda consistente *al final*, no inmediatamente.
- **Entregas duplicadas**: con garantía *at-least-once* un mensaje puede llegar dos veces (por ejemplo, si el ACK se pierde). Los consumidores deben ser **idempotentes**.
- **Orden**: una cola es FIFO, pero con varios consumidores, reintentos o requeue, el orden de procesamiento no está garantizado globalmente.
- **Depuración más difícil**: un flujo atraviesa varios servicios en momentos distintos; se necesitan logs con identificadores de correlación y monitoreo de colas.
- **Infraestructura adicional**: hay que operar, asegurar y monitorear el broker (disponibilidad, disco, memoria).
- **Contratos de mensajes**: el formato del evento es un contrato entre equipos; cambiarlo sin cuidado rompe a los consumidores (versionado de eventos).

3. Cuándo usar mensajería y cuándo no

La pregunta clave es: **¿quien inicia la operación necesita la respuesta para continuar?** Si la necesita, lo natural es una llamada síncrona. Si no, la mensajería suele ser mejor.

Usar mensajería cuando…	Preferir llamada síncrona (HTTP) cuando…
La tarea es secundaria o puede ocurrir después: enviar correos, generar boletas, PDFs, notificaciones, actualizar reportes.	El usuario necesita la respuesta ahora: consultar el catálogo, ver el carrito, validar stock antes de cobrar.
La tarea es lenta o depende de sistemas externos poco confiables (SMTP, pasarelas, APIs de terceros).	La operación es rápida y el resultado decide el siguiente paso del flujo.
Varios servicios deben reaccionar al mismo hecho (un evento con muchos interesados).	Hay un único destinatario conocido y la interacción es una consulta (query).
Hay picos de carga que conviene absorber y procesar de forma gradual.	Se requiere consistencia fuerte e inmediata entre los datos de ambos servicios.
Se quiere que la caída de un servicio no impida que otro siga funcionando.	La complejidad de un broker no se justifica (sistemas pequeños, un solo equipo).

> **En Pedidos360** se combinan ambos estilos: el checkout valida stock y carrito con llamadas HTTP (necesita la respuesta para cobrar) y, una vez confirmado el pago, publica un evento para el trabajo posterior (registrar el envío y notificar por correo), que no debe frenar la respuesta al cliente.

4. RabbitMQ y el modelo AMQP

RabbitMQ es un broker de mensajería de código abierto que implementa el protocolo **AMQP 0-9-1** (Advanced Message Queuing Protocol). Su característica distintiva es que el productor **no publica directamente en una cola**, sino en un **exchange**, que decide a qué colas enviar cada mensaje según reglas llamadas **bindings**. Esto separa *quién publica* de *quién recibe*.

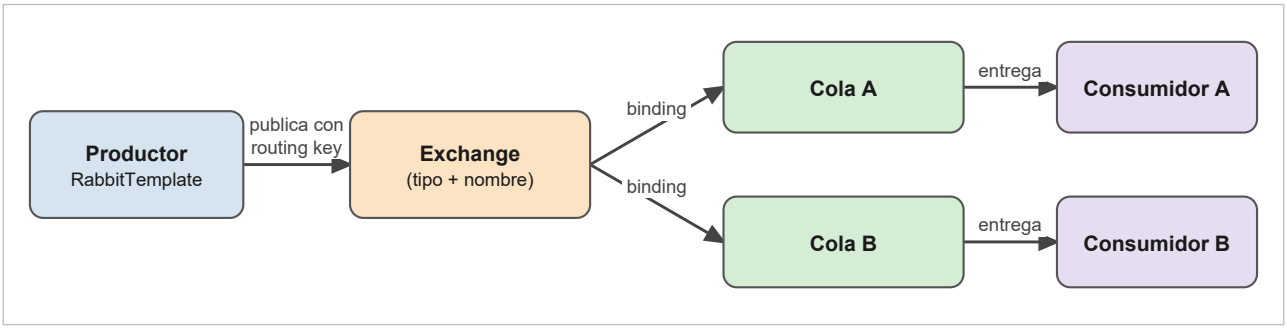


Figura 2. Modelo AMQP: el productor publica en un exchange; los bindings deciden a qué colas llega el mensaje.

4.1 Componentes

Elemento	Descripción
Conexión	Conexión TCP entre la aplicación y el broker (puerto 5672). Es costosa; se reutiliza.
Canal (channel)	Conexión virtual dentro de la conexión TCP. Cada operación (publicar, consumir, ACK) ocurre en un canal. Spring gestiona un pool de canales con *CachingConnectionFactory*.
Virtual host (vhost)	Espacio aislado de exchanges, colas y permisos dentro del mismo broker (por defecto */*).
Exchange	Punto de entrada de los mensajes. Tiene un **tipo** (direct, fanout, topic, headers) que define cómo enruta.

Elemento	Descripción
Cola	Almacena mensajes hasta que se consumen. Puede ser **durable** (sobrevive a reinicios del broker), exclusiva o autoeliminable, y recibir **argumentos** (TTL, dead-letter, largo máximo…).
Binding	Regla que une un exchange con una cola, normalmente con una **routing key** o un patrón.
Routing key	Etiqueta que el productor pone en el mensaje; el exchange la compara con los bindings.
Mensaje	Cuerpo (bytes) + propiedades (content_type, delivery_mode, message_id, correlation_id…) + headers.
Delivery tag	Número que identifica una entrega dentro de un canal; se usa para el ACK/NACK manual.
Prefetch	Cuántos mensajes sin confirmar puede tener un consumidor a la vez (en Spring AMQP, 250 por defecto).

4.2 Tipos de exchange

Tipo	Cómo enruta	Uso típico
direct	Entrega a las colas cuyo binding tiene **exactamente** la misma routing key. Si varias colas usan la misma clave, cada una recibe una copia.	Comandos o eventos con destino claro. **Es el que usa Pedidos360** (*pedidos.exchange*).
fanout	Ignora la routing key y entrega una copia a **todas** las colas ligadas.	Difusión (broadcast): invalidar cachés, notificar a todos los interesados.
topic	Compara la routing key con patrones separados por puntos: ***** reemplaza una palabra y **#** cero o más. Ej.: *pago.** recibe *pago.completado* y *pago.rechazado*.	Eventos jerárquicos con consumidores que filtran por categoría.
headers	Enruta según los headers del mensaje (x-match all/any) en vez de la routing key.	Reglas de enrutamiento con varios atributos; poco común.
default (sin nombre)	Exchange direct predefinido; toda cola está ligada a él con su propio nombre como routing key. Publicar en "" con clave *X* entrega directo a la cola *X*.	Enviar a una cola específica. Pedidos360 lo usa para reintentos y para la DLQ.

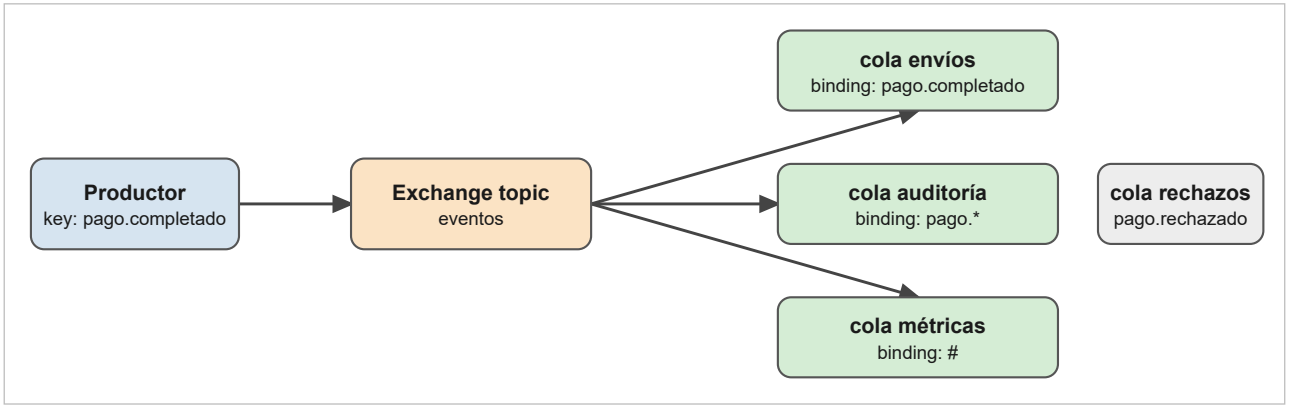


Figura 3. Exchange topic: un mismo evento llega a todas las colas cuyo patrón coincide; la cola de rechazos no lo recibe.

5. Garantías de entrega y confirmaciones

5.1 Semánticas de entrega

Semántica	Qué garantiza	Cómo se logra	Riesgo
At-most-once (como mucho una vez)	Nunca se procesa dos veces, pero puede perderse.	ACK automático: el broker borra el mensaje al entregarlo.	Si el consumidor falla a mitad, el mensaje se pierde.
At-least-once (al menos una vez)	Nunca se pierde, pero puede llegar repetido.	ACK manual después de procesar; mensajes y colas durables; confirmaciones del publicador.	Duplicados: el consumidor debe ser idempotente.
Exactly-once (exactamente una vez)	Se procesa una sola vez.	En la práctica: at-least-once + consumidor **idempotente** (efecto exactamente-una-vez).	No existe de forma nativa en RabbitMQ entre sistemas distintos.

Pedidos360 apunta a **at-least-once**: colas durables, mensajes persistentes (RabbitTemplate publica con *delivery_mode=persistent* por defecto) y ACK manual solo después de guardar el envío.

5.2 Confirmaciones del consumidor: ACK, NACK y reject

Operación	Efecto en el broker	Cuándo usarla
basicAck(tag, false)	El mensaje se procesó; se elimina de la cola.	Procesamiento exitoso.
basicNack(tag, false, true)	Rechaza y **re-encola**: vuelve a la cola para otro intento inmediato.	Casi nunca sin control: un error permanente generaría un bucle infinito (mensaje veneno).
basicNack(tag, false, false)	Rechaza **sin re-encolar**: si la cola tiene dead-letter exchange, va a la DLQ; si no, se descarta.	Error no recuperable o reintentos agotados.
basicReject(tag, requeue)	Igual que NACK pero para un solo mensaje (versión original de AMQP).	Equivalente a NACK individual.

En Spring AMQP, el modo de confirmación se define con *spring.rabbitmq.listener.simple.acknowledge-mode*:

- **AUTO** (por defecto en Spring): Spring hace ACK si el método termina sin excepción, y NACK si lanza una; por defecto re-encola (*default-requeue-rejected=true*), salvo excepciones fatales.
- **MANUAL**: el código recibe el *Channel* y el *delivery tag* y decide explícitamente ACK o NACK. **Es el modo que usa msvc-envio**, porque permite decidir entre reintentar, enviar a la DLQ o confirmar.
- **NONE**: ACK automático del broker al entregar (at-most-once).

5.3 Confirmaciones del publicador

El ACK del consumidor protege el tramo cola → consumidor. El tramo productor → broker se protege con:

- **Publisher confirms**: el broker confirma de forma asíncrona que recibió y guardó el mensaje (*spring.rabbitmq.publisher-confirm-type=correlated* + *ConfirmCallback*).
- **Mandatory + returns**: si ningún binding coincide, el broker devuelve el mensaje en vez de descartarlo en silencio (*spring.rabbitmq.template.mandatory=true* + *ReturnsCallback*).
- **Patrón Outbox** (sección 7): guarda el evento en la misma transacción que el dato de negocio y lo publica después, para no perder eventos si el broker está caído.

> **Pedidos360 usa publisher confirms y mandatory** en msvc-pago: después de publicar *pago.completado* espera hasta 5 s la confirmación del broker. Si RabbitMQ rechaza el mensaje, no responde o el mensaje no llega a ninguna cola, la compra igual se confirma (el pago ya se cobró) y el evento completo queda en los logs para reprocesarlo. No usa el patrón Outbox, así que ese reproceso es manual (ver la sección 13).

6. Manejo de errores: reintentos, DLX y DLQ

6.1 Errores recuperables y no recuperables

No todos los errores son iguales, y tratarlos igual causa problemas: re-encolar un error permanente genera un bucle infinito; mandar a la DLQ un error temporal pierde mensajes que se podrían haber procesado un minuto después.

Tipo	Ejemplos	Tratamiento correcto
Recuperable (transitorio)	Base de datos caída o saturada, timeout de red, servicio externo temporalmente no disponible.	Reintentar **con espera** y un **límite** de intentos. Si se agotan, DLQ.
No recuperable (permanente)	JSON inválido, campos obligatorios ausentes, violación de reglas de negocio, error de programación.	No reintentar: NACK sin requeue directo a la DLQ y registrar en logs.

6.2 Dead Letter Exchange (DLX) y Dead Letter Queue (DLQ)

Una cola puede declararse con los argumentos **x-dead-letter-exchange** y opcionalmente **x-dead-letter-routing-key**. Cuando un mensaje "muere" en esa cola, RabbitMQ lo reenvía automáticamente a ese exchange con esa clave, y desde ahí llega a la **DLQ**. Un mensaje muere cuando:

- **rejected**: el consumidor hace NACK/reject con *requeue=false*.
- **expired**: vence su TTL (por mensaje o por cola, *x-message-ttl*).
- **maxlen**: la cola superó su largo máximo (*x-max-length*).
- **delivery_limit**: en colas quorum, superó el número máximo de entregas.

Al hacer dead-lettering, RabbitMQ agrega el header **x-death**: una lista con la historia del mensaje (cola de origen, motivo, cantidad de veces, exchange y routing keys). La primera entrada es la más reciente. Además agrega *x-first-death-queue*, *x-first-death-reason* y, en versiones recientes, *x-last-death-reason*.

6.3 Estrategias de reintento

Estrategia	Cómo funciona	Ventajas	Desventajas
Requeue inmediato	NACK con requeue=true.	Simple.	Sin espera ni límite: bucles y CPU al 100 % ante errores permanentes.
Reintento en memoria	Spring Retry (RetryInterceptor) reintenta dentro del mismo hilo con backoff.	Fácil de configurar.	Bloquea al consumidor durante las esperas; se pierde el conteo si el servicio se reinicia.
Cola de espera con TTL + DLX	El mensaje va a una cola sin consumidores con TTL; al vencer, su DLX lo devuelve a la cola principal.	No bloquea al consumidor; sobrevive a reinicios; es visible en el broker.	Más colas que declarar; el retardo es fijo por cola.

Estrategia	Cómo funciona	Ventajas	Desventajas
Backoff exponencial	Varias colas de espera (10 s, 1 min, 10 min) o el plugin *delayed-message-exchange*.	Da tiempo creciente a que el sistema se recupere.	Más complejidad.

Pedidos360 usa la cola de espera con TTL + DLX en cada caso de uso: por ejemplo, *envio.reintento.queue* (TTL de 10 s) devuelve el mensaje a *envio.queue* mediante el exchange por defecto, y el número de reintento viaja en el header *x-numero-reintento*. Tras 3 reintentos el mensaje va a la DLQ del caso de uso. La lógica está una sola vez, en *ConsumidorConReintentosYDeadLetterQueue*.

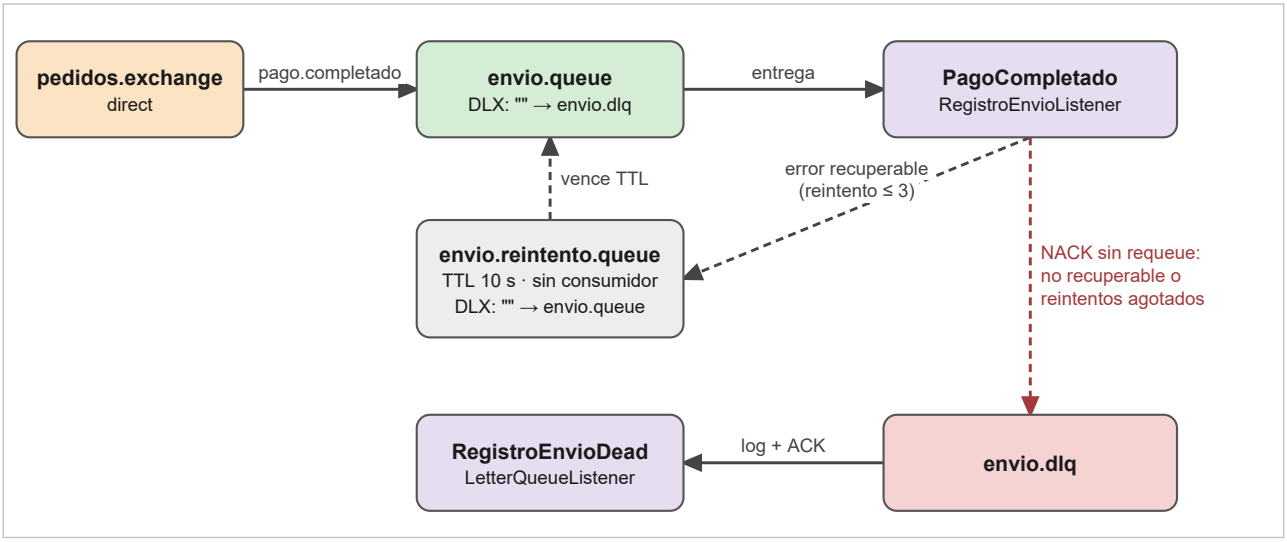


Figura 4. Reintentos y DLQ del registro de envíos; el mismo patrón se repite en cada caso de uso.

7. Arquitecturas y patrones de mensajería

Los patrones de mensajería fueron catalogados en *Enterprise Integration Patterns* (Hohpe y Woolf, 2003) y siguen vigentes. A continuación, los más importantes, para qué sirven y cuándo escogerlos.

7.1 Patrones de comunicación

Patrón	Descripción	Cuándo escogerlo	En RabbitMQ
Punto a punto / Work queue	Cada mensaje lo procesa **un solo** consumidor.	Repartir tareas (procesar pagos, generar PDFs).	Una cola con uno o varios consumidores.
Competing consumers	Varias instancias del mismo consumidor leen la misma cola y se reparten la carga.	Escalar el procesamiento horizontalmente.	Varias réplicas del servicio con el mismo @RabbitListener; ajustar prefetch.
Publish / Subscribe	Cada mensaje lo recibe **cada** suscriptor interesado.	Un hecho con varios interesados (envío, correo, auditoría).	Una cola por suscriptor ligada a un exchange fanout, topic o direct con la misma clave.
Routing / Topic	Los suscriptores eligen qué mensajes reciben por clave o patrón.	Familias de eventos (pago.completado, pago.rechazado).	Exchange direct o topic.
Request / Reply (RPC)	El productor espera una respuesta por una cola de respuesta con *correlation_id*.	Rara vez: si se necesita respuesta, HTTP suele ser más simple.	reply_to + correlation_id; *convertSendAndReceive* de RabbitTemplate.

7.2 Tipos de mensaje: comando, evento y consulta

- **Comando**: una orden dirigida a un destinatario ("*GenerarBoleta*"). Nombre en imperativo; se espera que alguien lo ejecute.
- **Evento**: un hecho que ya ocurrió ("*PagoCompletado*"). Nombre en pasado; el emisor no sabe ni le importa quién reacciona. **Pedidos360 publica eventos.**
- **Consulta**: una pregunta que espera respuesta. Normalmente conviene hacerla síncrona.

Entre los eventos se distinguen dos estilos. En la **notificación de evento** el mensaje lleva solo un identificador y el consumidor debe consultar al emisor para obtener los datos (acopla de nuevo a ambos). En la **transferencia de estado por evento** (event-carried state transfer) el mensaje trae todos los datos necesarios. *PagoCompletadoEvent* sigue este segundo estilo: incluye pedido, usuario, correo, dirección, total e ítems, de modo que msvc-envio no necesita llamar a msvc-pago.

7.3 Patrones de confiabilidad

Patrón	Problema que resuelve	Cómo funciona
Dead Letter Channel (DLQ)	Mensajes que no se pueden procesar bloquean la cola o se pierden.	Se apartan a una cola especial para revisarlos, reprocesarlos o descartarlos.
Retry con espera	Errores temporales.	Reintento diferido y limitado (sección 6.3).
Idempotent consumer	Duplicados por entrega at-least-once.	El consumidor recuerda qué mensajes procesó (por id) o hace operaciones naturalmente idempotentes (*upsert*, restricción única).
Transactional Outbox	Guardar en la base y publicar en el broker no es atómico: uno puede fallar.	Se guarda el evento en una tabla *outbox* en la misma transacción que el dato de negocio; un proceso aparte lo publica y lo marca como enviado.
Inbox	Garantizar procesamiento único del lado consumidor.	El consumidor registra el id del mensaje en una tabla *inbox* en la misma transacción que su efecto.
Poison message / parking lot	Un mensaje que siempre falla y se reintenta sin fin.	Límite de entregas y apartarlo a una cola de "estacionamiento" para revisión manual.
Claim check	Mensajes muy grandes (archivos, imágenes).	Se guarda el contenido en un almacenamiento (S3) y el mensaje lleva solo la referencia.
Correlation identifier	Seguir un flujo a través de varios servicios.	Un id (correlation_id) viaja en los headers y aparece en todos los logs.

7.4 Coordinación entre servicios: coreografía, orquestación y sagas

Cuando un proceso de negocio abarca varios servicios (pagar → descontar stock → registrar envío → notificar), hay dos formas de coordinarlo:

- **Coreografía**: cada servicio reacciona a eventos y publica los suyos; no hay un coordinador central. Es muy desacoplada, pero el flujo completo no está escrito en ningún lugar. **Pedidos360 usa coreografía**: msvc-envio reacciona a *pago.completado*.
- **Orquestación**: un servicio orquestador envía comandos a cada participante y decide el siguiente paso. El flujo es explícito y fácil de seguir, pero el orquestador concentra lógica.
- **Saga**: una transacción de negocio larga dividida en pasos locales; si un paso falla, se ejecutan **acciones compensatorias** (por ejemplo, devolver el stock y anular el pago). Puede implementarse con coreografía o con orquestación.

7.5 Por qué RabbitMQ: comparación con otras alternativas

Tecnología	Modelo	Fortalezas	Cuándo escogerla
RabbitMQ	Broker de colas con exchanges y enrutamiento flexible; el mensaje se borra al confirmarse.	Enrutamiento rico, ACK por mensaje, DLX, TTL, prioridades, baja latencia, fácil de operar. Disponible administrado en AWS (Amazon MQ).	Tareas y eventos entre microservicios, colas de trabajo, enrutamiento por tipo de mensaje. **Caso de Pedidos360.**
Apache Kafka	Log distribuido y particionado; los mensajes se retienen y pueden releerse.	Rendimiento muy alto, replay de eventos, orden por partición, streaming.	Grandes volúmenes, analítica en tiempo real, event sourcing, varios consumidores que releen historia.
Amazon SQS / SNS	Colas (SQS) y pub/sub (SNS) totalmente administrados.	Sin servidores que operar, escala automática, DLQ con redrive policy.	Sistemas 100 % en AWS que priorizan no administrar infraestructura.
Redis Streams / Pub-Sub	Estructuras en memoria.	Muy simple y rápido.	Casos ligeros donde la durabilidad no es crítica.

Para este proyecto, RabbitMQ es una elección adecuada: el volumen es moderado, se necesitan colas de trabajo con confirmación individual, reintentos y DLQ, y la rúbrica pide explícitamente colas, exchanges, bindings y DLQ, conceptos nativos de RabbitMQ.

7.6 Tipos de cola en RabbitMQ

- **Classic**: la cola tradicional; la usa Pedidos360. Suficiente con un solo nodo.
- **Quorum**: replicada con el algoritmo Raft entre varios nodos; recomendada para alta disponibilidad en producción. Soporta *delivery-limit* para mensajes veneno.
- **Streams**: log persistente con lectura repetible, similar a Kafka.
- **Lazy / largo máximo / prioridad**: argumentos que ajustan memoria, tamaño y orden de entrega.

8. Cómo programarlo con Spring Boot y Spring AMQP

Spring AMQP es la integración oficial de Spring con RabbitMQ. Con el starter, Spring Boot crea automáticamente la conexión, el *RabbitTemplate* para publicar, el *RabbitAdmin* que declara exchanges, colas y bindings al arrancar, y el contenedor que ejecuta los *@RabbitListener*.

8.1 Paso 1: dependencia

pom.xml (msvc-pago, msvc-envio y msvc-rabbitmq-admin)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

8.2 Paso 2: conexión y nombres en application.yml

Los datos de conexión vienen de variables de entorno, y los nombres de exchanges, colas y routing keys se centralizan en el mismo archivo para no dejar valores sueltos en el código.

msvc-envio/src/main/resources/application.yml (extracto)

```
  rabbitmq:
    host: ${RABBITMQ_HOST:localhost}
    port: ${RABBITMQ_PORT:5672}
    username: ${RABBITMQ_USER:guest}
    password: ${RABBITMQ_PASSWORD:guest}
    listener:
      simple:
        acknowledge-mode: manual
...
rabbitmq:
  exchange:
    pedidos: pedidos.exchange
  queue:
    # Caso de uso 1: registrar el envío de cada pago completado.
    envio: envio.queue
    envio-reintento: envio.reintento.queue
    envio-dlq: envio.dlq
    # Caso de uso 2: enviar el correo de confirmación de cada pago completado.
    notificacion-correo: notificacion.correo.queue
    notificacion-correo-reintento: notificacion.correo.reintento.queue
    notificacion-correo-dlq: notificacion.correo.dlq
  routing-key:
    pago-completado: pago.completado
    notificacion-correo-enviada: notificacion.correo.enviada
  reintento:
    espera-milisegundos: 10000
    maximo-reintentos: 3
    header-numero-reintento: x-numero-reintento
```

8.3 Paso 3: declarar la topología con beans

Cada *@Bean* de tipo *Exchange*, *Queue* o *Binding* se declara en RabbitMQ al arrancar la aplicación. La declaración es idempotente: si el recurso ya existe con los mismos argumentos no pasa nada; si existe con argumentos **distintos**, RabbitMQ la rechaza (PRECONDITION_FAILED).

Ejemplo genérico de configuración

```
@Bean
public Queue miCola() {
    return QueueBuilder.durable("mi.cola")          // sobrevive a reinicios
            .deadLetterExchange("mi.dlx")          // a dónde van los mensajes muertos
            .deadLetterRoutingKey("mi.cola.dlq")
            .ttl(60_000)                           // opcional: vencimiento de mensajes
            .build();
}

@Bean
public DirectExchange miExchange() { return new DirectExchange("mi.exchange", true, false); }
@Bean
public Binding miBinding(Queue miCola, DirectExchange miExchange) {
    return BindingBuilder.bind(miCola).to(miExchange).with("mi.routing.key");
}

@Bean
public MessageConverter jsonMessageConverter() { return new Jackson2JsonMessageConverter(); }
```

8.4 Paso 4: publicar (productor)

Publicación con RabbitTemplate

```
rabbitTemplate.convertAndSend("mi.exchange", "mi.routing.key", evento);   // objeto -> JSON

// con headers adicionales
rabbitTemplate.convertAndSend("mi.exchange", "mi.routing.key", evento, mensaje -> {
    mensaje.getMessageProperties().setHeader("x-mi-header", 1);
    return mensaje;
});
```

8.5 Paso 5: consumir (consumidor)

Consumidor con ACK manual (requiere acknowledge-mode: manual)

```
@RabbitListener(queues = "${rabbitmq.queue.mi-cola}")
public void onMensaje(MiEvento evento, Channel channel,
                      @Header(AmqpHeaders.DELIVERY_TAG) long tag) throws IOException {
    try {
        procesar(evento);
        channel.basicAck(tag, false);              // éxito: se borra de la cola
    } catch (ErrorTemporal e) {
        // reintentar con espera (cola de reintento) o NACK con requeue controlado
    } catch (Exception e) {
        channel.basicNack(tag, false, false);      // sin requeue: va a la DLQ
    }
}
```

8.6 Paso 6: administración programática

RabbitAdmin permite declarar y borrar recursos en tiempo de ejecución, que es lo que hace msvc-rabbitmq-admin detrás de su API REST: *declareQueue*, *deleteQueue*, *declareExchange*, *deleteExchange*, *declareBinding* y *removeBinding*.

8.7 Buenas prácticas

- Centralizar nombres en *application.yml* y leerlos con *@Value*; nunca escribirlos en clases de negocio.
- Separar la configuración (clases *@Configuration*) de los productores, de los consumidores y del negocio.
- Usar JSON con un DTO/record por evento; tratar el evento como un contrato versionado.
- Declarar colas durables y mensajes persistentes cuando el mensaje no se puede perder.
- ACK manual cuando hay que distinguir errores; siempre con un límite de reintentos.
- Toda cola de trabajo debería tener su DLQ y alguien (o algo) que la revise.
- Consumidores idempotentes: asumir que un mensaje puede llegar dos veces.
- Registrar en logs el identificador del negocio (número de pedido) en cada paso.
- Probar con un RabbitMQ real (Docker o Testcontainers) los casos de éxito, error y DLQ.

9. RabbitMQ en Pedidos360: la topología completa

Pedidos360 tiene **tres casos de uso** de mensajería, todos sobre el exchange direct *pedidos.exchange*:

Caso de uso	Evento (routing key)	Productor → consumidor	Qué hace
1. Registro del envío	pago.completado	msvc-pago → msvc-envio PagoCompletadoRegistroEnvioListener	Guarda el envío del pedido en estado PREPARANDO.
2. Notificación por correo	pago.completado	msvc-pago → msvc-envio PagoCompletadoNotificacionCorreoListener	Envía el comprobante por SMTP y registra el resultado.
3. Confirmación del correo	notificacion.correo.enviada	msvc-envio → msvc-pago NotificacionCorreoEnviadaListener	Marca el pedido con *emailEnviado = true*.

Los casos 1 y 2 se ligan con la **misma** routing key: el exchange direct entrega una copia del evento a cada cola (publish/subscribe). Así, un correo que falla se reintenta por su cuenta sin afectar al registro del envío. El caso 3 cierra el ciclo: msvc-pago es productor y también consumidor. Además, **msvc-rabbitmq-admin** permite administrar colas, exchanges y bindings por API REST.

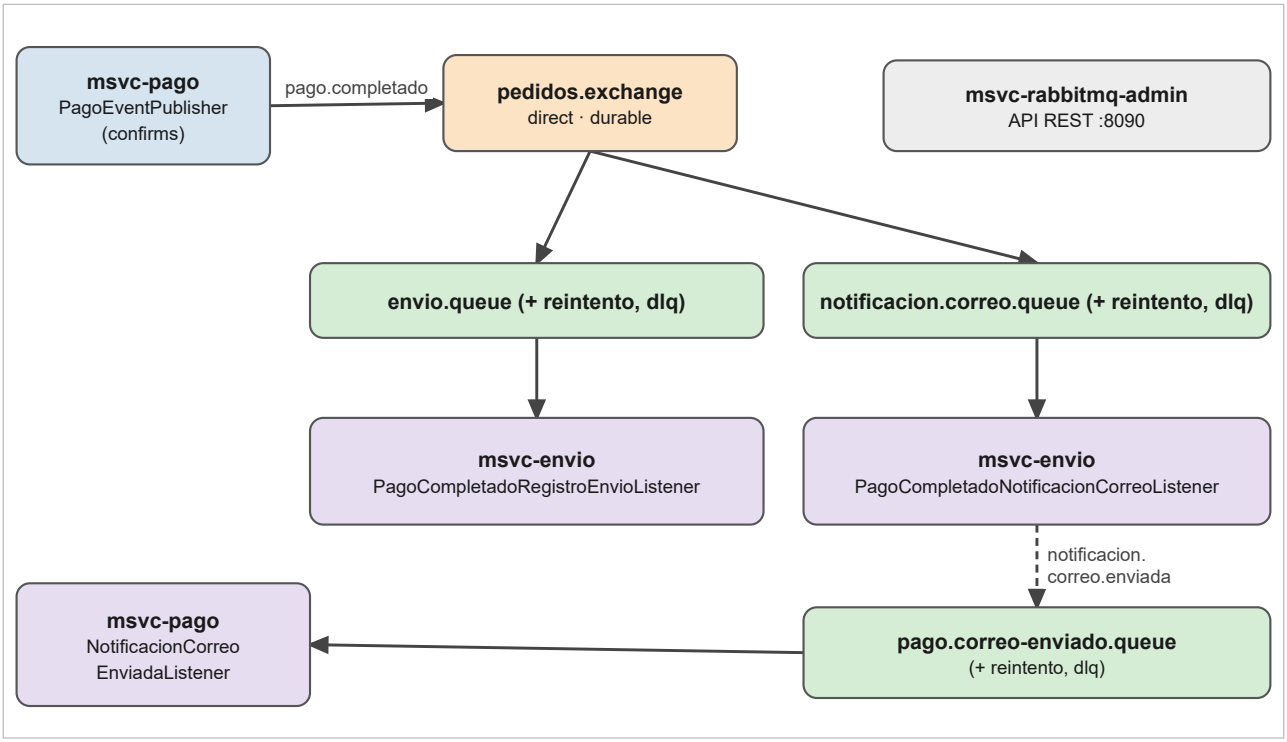


Figura 5. Topología RabbitMQ de Pedidos360: tres casos de uso, cada uno con su cola, su cola de reintento y su DLQ.

9.1 Inventario de recursos

Recurso	Tipo y propiedades	Declarado en	Para qué existe
pedidos.exchange	Exchange **direct**, durable.	RabbitMQConfig de msvc-pago **y** de msvc-envio.	Punto de entrada de todos los eventos. Ambos lo declaran para que cualquiera pueda arrancar primero.
envio.queue · **notificacion.correo.queue** · **pago.correo-enviado.queue**	Colas durables con *x-dead-letter-exchange=""* y *x-dead-letter-routing-key* = su DLQ.	RegistroEnvioMessagingConfig, NotificacionCorreoMessagingConfig, NotificacionCorreoEnviadaMessagingConfig.	Cola de trabajo de cada caso de uso; sus mensajes muertos van a su DLQ por el exchange por defecto.
***.reintento.queue** (tres)	Colas durables **sin consumidores**; *x-message-ttl=10000* y dead-letter hacia su cola principal.	Las mismas tres clases.	Sala de espera de los reintentos: al vencer los 10 s el mensaje vuelve a la cola principal.
envio.dlq · **notificacion.correo.dlq** · **pago.correo-enviado.dlq**	Colas durables sin argumentos.	Las mismas tres clases.	Destino de los mensajes no procesables; cada una la lee un consumidor que los registra en logs.
Bindings	*pago.completado* → envio.queue y notificacion.correo.queue; *notificacion.correo.enviada* → pago.correo-enviado.queue.	Las mismas tres clases.	Conectan cada evento con las colas interesadas.
Exchange por defecto ("")	Direct predefinido por RabbitMQ.	No se declara.	Entrega directa a una cola por su nombre; se usa para DLQ y reintentos.
x-numero-reintento	Header numérico del mensaje.	ReintentoDeMensajesPublisher.	Cuenta cuántos reintentos lleva un mensaje.

Recurso	Tipo y propiedades	Declarado en	Para qué existe
jsonMessageConverter / **rawMessageConverter**	Jackson2JsonMessageConverter / SimpleMessageConverter (*defaultCandidate=false*).	RabbitMQConfig de cada servicio.	JSON para los eventos; sin conversión para que los consumidores de DLQ lean incluso JSON inválido.

9.2 Parámetros configurables

Propiedad	Valor	Servicio	Significado
spring.rabbitmq.host / port	${RABBITMQ_HOST:localhost} / 5672	todos	Dirección del broker. En Docker, RABBITMQ_HOST=rabbitmq (nombre del servicio en la red de compose).
spring.rabbitmq.listener.simple.acknowledge-mode	manual	pago, envio	El código decide ACK o NACK de cada mensaje.
spring.rabbitmq.publisher-confirm-type / publisher-returns / template.mandatory	correlated / true / true	pago	El broker confirma cada publicación y devuelve las que no llegan a ninguna cola.
rabbitmq.publicacion.espera-confirmacion-milisegundos	5000	pago	Tiempo máximo de espera de la confirmación.
rabbitmq.exchange.pedidos	pedidos.exchange	pago, envio	Nombre del exchange.
rabbitmq.routing-key.*	pago.completado, notificacion.correo.enviada	pago, envio	Claves de los eventos.
rabbitmq.queue.*	nombres de las nueve colas	pago, envio	Cola principal, de reintento y DLQ de cada caso.
rabbitmq.reintento.espera-milisegundos / maximo-reintentos	10000 / 3	pago, envio	TTL de las colas de reintento y reintentos antes de la DLQ.
rabbitmq.reintento.header-numero-reintento	x-numero-reintento	pago, envio	Nombre del header contador.

9.3 Infraestructura: el servicio rabbitmq en Docker Compose

pedidos-backend/docker-compose.yml (entorno local)

```
# Broker de mensajería para comunicación asíncrona entre microservicios.
rabbitmq:
  image: rabbitmq:3-management
  ports:
    - "5672:5672"
    - "15672:15672"
  healthcheck:
    # Se ejecuta como el usuario rabbitmq: como root, el chequeo puede crear .erlang.cookie antes que
    # el broker y dejarlo sin permiso de lectura, con lo que RabbitMQ no arranca la primera vez.
    test: ["CMD", "gosu", "rabbitmq", "rabbitmq-diagnostics", "-q", "check_port_connectivity"]
    interval: 10s
    timeout: 5s
    retries: 10
  volumes:
    - rabbitmq_data:/var/lib/rabbitmq
```

- **Imagen rabbitmq:3-management**: incluye la consola web de administración (puerto 15672) y su API HTTP.
- **Puertos**: 5672 es AMQP (lo usan los microservicios); 15672 es la consola web. En local se publican para poder usar la consola en *http://localhost:15672* (usuario y clave *guest*).
- **Healthcheck**: *rabbitmq-diagnostics check_port_connectivity* comprueba que el broker acepta conexiones. Se ejecuta con *gosu rabbitmq* porque, como root, el chequeo podía crear el archivo

.erlang.cookie antes que el broker y dejarlo ilegible, impidiendo el primer arranque.

- **depends_on con condition: service_healthy**: msvc-envio, msvc-pago y msvc-rabbitmq-admin esperan a que RabbitMQ esté sano antes de arrancar.
- **Volumen rabbitmq_data**: conserva colas durables y mensajes persistentes entre reinicios del contenedor.
- **En AWS** (*docker-compose.prod.yml* de la EC2 backend) RabbitMQ no publica puertos: solo es accesible dentro de la red Docker de la instancia privada.

10. El código, servicio por servicio

10.1 Organización de los paquetes

Servicio	Paquete	Responsabilidad
msvc-pago	config	RabbitMQConfig (exchange y conversores) y NotificacionCorreoEnviadaMessagingConfig (caso 3).
msvc-pago	service	PagoEventPublisher (productor con confirms) y ReintentoDeMensajesPublisher.
msvc-pago	listener / listener.notificacioncorreo	Clase base de los consumidores y consumidores del caso 3.
msvc-envio	config	RabbitMQConfig, RegistroEnvioMessagingConfig (caso 1) y NotificacionCorreoMessagingConfig (caso 2).
msvc-envio	listener.registroenvio	PagoCompletadoRegistroEnvioListener y RegistroEnvioDeadLetterQueueListener.
msvc-envio	listener.notificacioncorreo	PagoCompletadoNotificacionCorreoListener y NotificacionCorreoDeadLetterQueueListener.
msvc-envio	publisher / service	Publicadores (reintento y aviso de correo enviado) y CorreoConfirmacionCompraService (arma y envía el correo; no conoce RabbitMQ).
msvc-rabbitmq-admin	controller / service / dto	API REST, RabbitAdminService y DTO con validación.

10.2 msvc-pago: el productor con confirmaciones

msvc-pago · service/PagoEventPublisher.java

```
/**
 * Publica eventos de pago completado en el exchange de RabbitMQ.
 * Mantiene la lógica de mensajería separada del controlador.
 *
 * Espera la confirmación del broker (publisher confirms): si RabbitMQ no confirma, o el mensaje no llega a
    ninguna
 * cola (mandatory), lanza PublicacionDeEventoFallidaException en vez de perder el evento en silencio.
 */
@Service
public class PagoEventPublisher {

    private static final Logger log = LoggerFactory.getLogger(PagoEventPublisher.class);
```

```
    private final RabbitTemplate rabbitTemplate;
    private final String exchangeName;
    private final String routingKey;
    private final long esperaConfirmacionMilisegundos;

    public PagoEventPublisher(
            RabbitTemplate rabbitTemplate,
            @Value("${rabbitmq.exchange.pedidos}") String exchangeName,
            @Value("${rabbitmq.routing-key.pago-completado}") String routingKey,
            @Value("${rabbitmq.publicacion.espera-confirmacion-milisegundos}") long
                esperaConfirmacionMilisegundos
    ) {
        this.rabbitTemplate = rabbitTemplate;
        this.exchangeName = exchangeName;
        this.routingKey = routingKey;
        this.esperaConfirmacionMilisegundos = esperaConfirmacionMilisegundos;
    }

    public void publicarPagoCompletado(PagoCompletadoEvent event) {
        log.info("Publicando evento pago.completado para pedido: {}", event.numeroPedido());
        CorrelationData correlationData = new CorrelationData(event.pedidoId().toString());
        rabbitTemplate.convertAndSend(exchangeName, routingKey, event, correlationData);
        CorrelationData.Confirm confirmacion = esperarConfirmacion(correlationData, event.numeroPedido());
        if (correlationData.getReturned() != null) {
            throw new PublicacionDeEventoFallidaException("Ninguna cola recibió el evento del pedido "
                    + event.numeroPedido() + ": " + correlationData.getReturned().getReplyText());
        }
        if (!confirmacion.ack()) {
            throw new PublicacionDeEventoFallidaException("RabbitMQ rechazó el evento del pedido "
                    + event.numeroPedido() + ": " + confirmacion.reason());
        }
        log.info("Evento confirmado por RabbitMQ para pedido: {}", event.numeroPedido());
    }
    private CorrelationData.Confirm esperarConfirmacion(CorrelationData correlationData, String
        numeroPedido) {
        try {
            return correlationData.getFuture().get(esperaConfirmacionMilisegundos, TimeUnit.MILLISECONDS);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new PublicacionDeEventoFallidaException("Se interrumpió la espera de confirmación del
                pedido " + numeroPedido, e);
        } catch (ExecutionException | TimeoutException e) {
            throw new PublicacionDeEventoFallidaException("RabbitMQ no confirmó el evento del pedido " +
                numeroPedido, e);
        }
    }
}
```

El evento transporta todos los datos que los consumidores necesitan (transferencia de estado por evento):

msvc-pago · event/PagoCompletadoEvent.java

```
/**
 * Evento publicado a RabbitMQ cuando un pago se completa exitosamente.
 * Contiene toda la información que msvc-envio necesita para procesar el envío
 * sin hacer llamadas HTTP de vuelta a msvc-pago.
 */
public record PagoCompletadoEvent(
        UUID pedidoId,
        String numeroPedido,
        UUID usuarioId,
        String nombre,
        String email,
        String direccion,
        long total,
        List<ItemEvent> items
) {
    public record ItemEvent(String nombre, int cantidad, long precio, String imagenUrl) {
    }
}
```

En el checkout, después de guardar el pedido, se publica el evento. Si la publicación falla, la compra no se revierte:

msvc-pago · controller/PagoController.java (extracto del checkout)

```
        // Publicar evento asíncrono a RabbitMQ para que msvc-envio procese el envío/email.
        // El checkout responde inmediatamente sin esperar al correo.
        List<PagoCompletadoEvent.ItemEvent> eventItems = emailItems.stream()
                .map(i -> new PagoCompletadoEvent.ItemEvent(i.nombre(), i.cantidad(), i.precio(),
                    i.imagenUrl()))
                .toList();

        PagoCompletadoEvent event = new PagoCompletadoEvent(
                pedido.getId(),
                pedido.getNumeroPedido(),
                userId,
                nombre,
                email,
                request.direccion(),
                total,
                eventItems
        );

        try {
            pagoEventPublisher.publicarPagoCompletado(event);
        } catch (Exception e) {
            // Si RabbitMQ está caído o no confirma, el pedido sigue siendo válido: el pago ya se cobró.
            // El evento queda registrado en logs con su contenido para reprocesarlo manualmente.
            log.error("No se publicó pago.completado para el pedido {}. Evento para reproceso manual: {}.
                Causa: {}",
                    pedido.getNumeroPedido(), event, e.getMessage());
        }
```

10.3 msvc-envio: configuración por caso de uso

msvc-envio · config/RabbitMQConfig.java (lo compartido)

```
/**
 * Configuración de RabbitMQ compartida por los casos de uso de msvc-envio: el exchange y los conversores.
 * Las colas y bindings de cada caso de uso están en su propia clase de configuración:
 *   - RegistroEnvioMessagingConfig: registrar el envío de cada pago completado.
 *   - NotificacionCorreoMessagingConfig: enviar el correo de confirmación de cada pago completado.
 */
@Configuration
public class RabbitMQConfig {

    /** Exchange por defecto de RabbitMQ: entrega directo a la cola cuyo nombre coincide con la routing
        key. */
    public static final String DEFAULT_EXCHANGE = "";
    public static final String RAW_MESSAGE_CONVERTER_BEAN_NAME = "rawMessageConverter";

    @Value("${rabbitmq.exchange.pedidos}")
    private String exchangeName;

    /** Exchange donde msvc-pago publica los eventos y donde msvc-envio publica la confirmación del correo.
        */
    @Bean
    public DirectExchange pedidosExchange() {
        return new DirectExchange(exchangeName, true, false);
    }
    /** Serializa y deserializa los eventos como JSON. */
    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }

    /**
     * Conversor sin transformación para los consumidores de las DLQ, que deben poder leer también mensajes
         con JSON inválido.
     * No es candidato por defecto para que el resto de la aplicación siga usando el conversor JSON.
     */
    @Bean(name = RAW_MESSAGE_CONVERTER_BEAN_NAME, defaultCandidate = false)
    public MessageConverter rawMessageConverter() {
        return new SimpleMessageConverter();
    }
}
```

msvc-envio · config/NotificacionCorreoMessagingConfig.java (caso 2; el caso 1 es análogo)

```
/**
 * Caso de uso 2: enviar el correo de confirmación de cada pago completado.
 * Recibe una copia del mismo evento que el registro de envío (publish/subscribe): un correo que falla
 * se reintenta por su cuenta sin afectar al registro del envío.
 *
 * Ruta de un mensaje:
 *   pedidos.exchange --(pago.completado)--> notificacion.correo.queue
 *   notificacion.correo.queue --(error recuperable)--> notificacion.correo.reintento.queue --(espera
     TTL)--> notificacion.correo.queue
 *   notificacion.correo.queue --(error no recuperable o reintentos agotados)--> notificacion.correo.dlq
 *   correo enviado --> pedidos.exchange --(notificacion.correo.enviada)--> msvc-pago
 */
@Configuration
public class NotificacionCorreoMessagingConfig {

    @Value("${rabbitmq.queue.notificacion-correo}")
    private String queueName;

    @Value("${rabbitmq.queue.notificacion-correo-reintento}")
    private String reintentoQueueName;

    @Value("${rabbitmq.queue.notificacion-correo-dlq}")
    private String dlqName;
    @Value("${rabbitmq.routing-key.pago-completado}")
    private String pagoCompletadoRoutingKey;

    @Value("${rabbitmq.reintento.espera-milisegundos}")
    private int reintentoEsperaMilisegundos;

    /** Cola principal de notificaciones; sus mensajes muertos van a notificacion.correo.dlq. */
    @Bean
    public Queue notificacionCorreoQueue() {
        return QueueBuilder.durable(queueName)
                .deadLetterExchange(DEFAULT_EXCHANGE)
                .deadLetterRoutingKey(dlqName)
                .build();
    }

    /** Cola de espera sin consumidores: al vencer el TTL, el mensaje vuelve a la cola de notificaciones. */
    @Bean
    public Queue notificacionCorreoReintentoQueue() {
        return QueueBuilder.durable(reintentoQueueName)
                .ttl(reintentoEsperaMilisegundos)
                .deadLetterExchange(DEFAULT_EXCHANGE)
                .deadLetterRoutingKey(queueName)
                .build();
    }
    /** Cola donde terminan los correos que no se pudieron enviar. */
    @Bean
    public Queue notificacionCorreoDlq() {
        return QueueBuilder.durable(dlqName).build();
    }

    /** Misma routing key que el registro de envío: el exchange direct entrega una copia a cada cola. */
    @Bean
    public Binding notificacionCorreoBinding(Queue notificacionCorreoQueue, DirectExchange pedidosExchange)
        {
        return BindingBuilder.bind(notificacionCorreoQueue).to(pedidosExchange).with(pagoCompletadoRoutingKe
            y);
    }
}
```

10.4 La clase base de los consumidores

Toda la lógica de confirmación vive en una sola clase abstracta. Cada consumidor solo define qué hace con el evento (*procesar*), qué errores son recuperables y cómo se identifica en los logs. Así las decisiones de ACK, reintento y DLQ son idénticas en todos los consumidores.

msvc-envio · listener/ConsumidorConReintentosYDeadLetterQueue.java

```
/**
 * Manejo de confirmaciones común a todos los consumidores de msvc-envio (ACK manual).
 *
 * Decisión ante cada mensaje:
 *   - Procesado sin errores: ACK, el mensaje se elimina de la cola.
 *   - Error recuperable (falla temporal de infraestructura) con reintentos disponibles: se publica una
     copia
 *     en la cola de reintento del caso de uso y se hace ACK del original; la copia vuelve tras la espera
     (TTL).
 *   - Error no recuperable (datos inválidos) o reintentos agotados: NACK sin requeue, el mensaje va a la
     DLQ.
 *
 * Cada consumidor concreto solo define qué hace con el evento y qué errores considera recuperables.
 */
public abstract class ConsumidorConReintentosYDeadLetterQueue<E> {

    protected final Logger log = LoggerFactory.getLogger(getClass());

    private final ReintentoDeMensajesPublisher reintentoPublisher;
    private final String reintentoQueueName;
    private final int maximoReintentos;
    private final String headerNumeroReintento;

    protected ConsumidorConReintentosYDeadLetterQueue(
            ReintentoDeMensajesPublisher reintentoPublisher,
            String reintentoQueueName,
            int maximoReintentos,
            String headerNumeroReintento
    ) {
        this.reintentoPublisher = reintentoPublisher;
        this.reintentoQueueName = reintentoQueueName;
        this.maximoReintentos = maximoReintentos;
        this.headerNumeroReintento = headerNumeroReintento;
    }

    /** Lógica de negocio del caso de uso. Debe lanzar una excepción si el mensaje no se pudo procesar. */
    protected abstract void procesar(E event) throws Exception;

    /** Indica si vale la pena reintentar: el mismo error podría no repetirse más tarde. */
    protected abstract boolean esErrorRecuperable(Exception e);

    /** Identificador de negocio que se muestra en los logs, por ejemplo el número de pedido. */
    protected abstract String describir(E event);

    protected final void procesarConConfirmacionManual(
            E event,
            Channel channel,
            long deliveryTag,
            Map<String, Object> headers
    ) throws IOException {
        int reintentosRealizados = leerReintentosRealizados(headers);
        String identificador = event == null ? "desconocido" : describir(event);
        log.info("Mensaje recibido para pedido: {} (reintentos realizados: {})", identificador,
            reintentosRealizados);

        try {
            procesar(event);
            channel.basicAck(deliveryTag, false);
            log.info("ACK enviado para pedido: {}", identificador);
        } catch (Exception e) {
```

```
            boolean recuperable = esErrorRecuperable(e);
            if (recuperable && reintentosRealizados < maximoReintentos) {
                reintentar(event, identificador, reintentosRealizados + 1, e, channel, deliveryTag);
            } else {
                log.error("Error {} procesando pedido {} tras {} reintentos. Enviando a DLQ: {}",
                        recuperable ? "recuperable" : "no recuperable", identificador,
                            reintentosRealizados, e.getMessage());
                channel.basicNack(deliveryTag, false, false);
            }
        }
    }
    /** Publica el evento en la cola de reintento y confirma el original; si no puede, lo deriva a la DLQ.
        */
    private void reintentar(
            E event,
            String identificador,
            int numeroReintento,
            Exception causa,
            Channel channel,
            long deliveryTag
    ) throws IOException {
        try {
            reintentoPublisher.publicarParaReintento(reintentoQueueName, event, numeroReintento);
        } catch (AmqpException errorAlPublicar) {
            log.error("No se pudo programar el reintento del pedido {}. Enviando a DLQ: {}",
                    identificador, errorAlPublicar.getMessage());
            channel.basicNack(deliveryTag, false, false);
            return;
        }
        // ACK del original: la copia en la cola de reintento volverá a la cola principal al vencer la
            espera
        channel.basicAck(deliveryTag, false);
        log.warn("Error recuperable procesando pedido {}. Programado reintento {} de {}: {}",
                identificador, numeroReintento, maximoReintentos, causa.getMessage());
    }
    private int leerReintentosRealizados
```

msvc-envio · publisher/ReintentoDeMensajesPublisher.java

```
/**
 * Publica en la cola de reintento de un caso de uso los eventos que fallaron por un error recuperable.
 * Mantiene la mensajería separada de los consumidores: estos solo deciden si corresponde reintentar.
 */
@Component
public class ReintentoDeMensajesPublisher {

    private final RabbitTemplate rabbitTemplate;
    private final String headerNumeroReintento;

    public ReintentoDeMensajesPublisher(
            RabbitTemplate rabbitTemplate,
            @Value("${rabbitmq.reintento.header-numero-reintento}") String headerNumeroReintento
    ) {
        this.rabbitTemplate = rabbitTemplate;
        this.headerNumeroReintento = headerNumeroReintento;
    }

    /** El número de reintento viaja en un header para que el consumidor sepa cuántos lleva. */
    public void publicarParaReintento(String reintentoQueueName, Object event, int numeroReintento) {
        rabbitTemplate.convertAndSend(DEFAULT_EXCHANGE, reintentoQueueName, event, message -> {
            message.getMessageProperties().setHeader(headerNumeroReintento, numeroReintento);
            return message;
        });
    }
}
```

10.5 Caso de uso 1: registro del envío

msvc-envio · listener/registroenvio/PagoCompletadoRegistroEnvioListener.java

```
/**
 * Caso de uso 1: registra el envío de cada pago completado.
 *
 * Errores:
 *   - Recuperables: fallas temporales de la base de datos (no responde, timeout). Se reintentan con espera.
 *   - No recuperables: evento sin los datos obligatorios. Van directo a la DLQ.
 * Es idempotente: si el pedido ya tiene un envío registrado, confirma el mensaje sin crear otro.
 */
@Component
public class PagoCompletadoRegistroEnvioListener extends
    ConsumidorConReintentosYDeadLetterQueue<PagoCompletadoEvent> {
    private final EnvioRepository envioRepository;

    public PagoCompletadoRegistroEnvioListener(
            EnvioRepository envioRepository,
            ReintentoDeMensajesPublisher reintentoPublisher,
            @Value("${rabbitmq.queue.envio-reintento}") String reintentoQueueName,
            @Value("${rabbitmq.reintento.maximo-reintentos}") int maximoReintentos,
            @Value("${rabbitmq.reintento.header-numero-reintento}") String headerNumeroReintento
    ) {
        super(reintentoPublisher, reintentoQueueName, maximoReintentos, headerNumeroReintento);
        this.envioRepository = envioRepository;
    }
    @RabbitListener(queues = "${rabbitmq.queue.envio}")
    public void onPagoCompletado(
            PagoCompletadoEvent event,
            Channel channel,
            @Header(AmqpHeaders.DELIVERY_TAG) long deliveryTag,
            @Headers Map<String, Object> headers
    ) throws IOException {
        procesarConConfirmacionManual(event, channel, deliveryTag, headers);
    }

    @Override
    protected void procesar(PagoCompletadoEvent event) {
        validarEvento(event);

        if (envioRepository.existsByPedidoId(event.pedidoId())) {
            log.info("El pedido {} ya tiene un envío registrado; mensaje duplicado", event.numeroPedido());
            return;
        }

        Envio envio = new Envio();
        envio.setPedidoId(event.pedidoId());
        envio.setNumeroPedido(event.numeroPedido());
        envio.setUsuarioId(event.usuarioId());
        envio.setEmail(event.email());
        envio.setDireccion(event.direccion());
        envio.setEstado("PREPARANDO");
        envio.setEmailEnviado(false);
        envioRepository.save(envio);
    }

    /** Son recuperables las fallas temporales de infraestructura; el resto se repetiría igual en cada
        intento. */
    @Override
    protected boolean esErrorRecuperable(Exception e) {
        return e instanceof TransientDataAccessException
                || e instanceof DataAccessResourceFailureException
                || e instanceof CannotCreateTransactionException;
    }

    @Override
    protected String describir(PagoCompletadoEvent event) {
        return event.numeroPedido();
    }

    /** Un evento sin los datos obligatorios
```

10.6 Caso de uso 2: notificación por correo

msvc-envio · listener/notificacioncorreo/PagoCompletadoNotificacionCorreoListener.java

```
/**
 * Caso de uso 2: envía el correo de confirmación de cada pago completado.
 *
 * Errores:
 *   - Recuperables: el servidor SMTP no responde (MailSendException) o la base de datos falla
    temporalmente.
 *   - No recuperables: evento sin datos obligatorios, credenciales SMTP rechazadas o correo mal construido.
 * Es idempotente: si el pedido ya tiene una notificación registrada, no vuelve a enviar el correo.
 * Si SMTP está deshabilitado, registra la notificación como omitida y confirma el mensaje.
 */
@Component
public class PagoCompletadoNotificacionCorreoListener extends
    ConsumidorConReintentosYDeadLetterQueue<PagoCompletadoEvent> {

    private final NotificacionCorreoRepository notificacionCorreoRepository;
    private final CorreoConfirmacionCompraService correoConfirmacionCompraService;
    private final NotificacionCorreoEnviadaPublisher notificacionCorreoEnviadaPublisher;
    private final boolean mailEnabled;

    public PagoCompletadoNotificacionCorreoListener(
            NotificacionCorreoRepository notificacionCorreoRepository,
            CorreoConfirmacionCompraService correoConfirmacionCompraService,
            NotificacionCorreoEnviadaPublisher notificacionCorreoEnviadaPublisher,
            ReintentoDeMensajesPublisher reintentoPublisher,
            @Value("${mail.enabled}") boolean mailEnabled,
            @Value("${rabbitmq.queue.notificacion-correo-reintento}") String reintentoQueueName,
            @Value("${rabbitmq.reintento.maximo-reintentos}") int maximoReintentos,
            @Value("${rabbitmq.reintento.header-numero-reintento}") String headerNumeroReintento
    ) {
        super(reintentoPublisher, reintentoQueueName, maximoReintentos, headerNumeroReintento);
        this.notificacionCorreoRepository = notificacionCorreoRepository;
        this.correoConfirmacionCompraService = correoConfirmacionCompraService;
        this.notificacionCorreoEnviadaPublisher = notificacionCorreoEnviadaPublisher;
        this.mailEnabled = mailEnabled;
    }
    @RabbitListener(queues = "${rabbitmq.queue.notificacion-correo}")
    public void onPagoCompletado(
            PagoCompletadoEvent event,
            Channel channel,
            @Header(AmqpHeaders.DELIVERY_TAG) long deliveryTag,
            @Headers Map<String, Object> headers
    ) throws IOException {
        procesarConConfirmacionManual(event, channel, deliveryTag, headers);
    }

    @Override
    protected void procesar(PagoCompletadoEvent event) {
        validarEvento(event);

        if (notificacionCorreoRepository.existsByPedidoId(event.pedidoId())) {
            log.info("El pedido {} ya fue notificado; mensaje duplicado", event.numeroPedido());
            return;
        }

        if (!mailEnabled) {
            registrarNotificacion(event, NotificacionCorreo.ESTADO_OMITIDA_SMTP_DESHABILITADO);
            log.info("SMTP deshabilitado: se omite el correo del pedido {}", event.numeroPedido());
            return;
        }
        correoConfirmacionCompraService.enviarConfirmacionDeCompra(event);
        registrarNotificacion(event, NotificacionCorreo.ESTADO_ENVIADA);
        log.info("Email enviado exitosamente para pedido: {}", event.numeroPedido());
        avisarCorreoEnviado(event);
    }

    /** Son recuperables las fallas temporales del servidor SMTP o de la base de datos. */
    @Override
    protected boolean esErrorRecuperable(Exception e) {
        return e instanceof MailSendException
                || e instanceof TransientDataAccessException
```

```
                || e instanceof DataAccessResourceFailureException
                || e instanceof CannotCreateTransactionException;
    }

    @Override
    protected String describir(PagoCompletadoEvent event) {
        return event.numeroPedido();
    }

    private void registrarNotificacion(PagoCompletadoEvent event, String estado) {
        NotificacionCorreo notificacion = new NotificacionCorreo();
        notificacion.setPedidoId(event.pedidoId());
        notificacion.setNumeroPedido(event.numeroPedido());
        notificacion.setEmail(event.email());
        notificacion.setEstado(estado);
        notificacionCorreoRepository.save(notificacion);
    }

    /** El correo ya salió: si el aviso a msvc-pago falla, se registra en logs pero no se reintenta el
        correo. */
    private void avisarCorreoEnviado(PagoCompletadoEvent event) {
        try {
            notificacionCorreoEnviadaPublisher.publicarCorreoEnviado(
                    new NotificacionCorreoEnviadaEvent(event.pedidoId(), event.numeroPedido()));
        } catch (AmqpException e) {
            log.error("No se pudo avisar a msvc-pago que se envió el correo del pedido {}: {}",
                    event.numeroPedido(), e.getMessage());
        }
    }

    /** Sin pedido, número o correo
```

- **Idempotencia**: la tabla *notificaciones_correo* tiene *pedidoId* único; si ya existe, el correo no se vuelve a enviar.
- **Errores del correo**: *MailSendException* (el servidor SMTP no responde) se reintenta; *MailAuthenticationException* o un correo mal construido van directo a la DLQ, porque reintentar no los arregla.
- **Aviso a msvc-pago**: si falla, se registra en logs pero no se reintenta, para no enviar el correo dos veces.

10.7 Caso de uso 3: msvc-pago como consumidor

msvc-pago · listener/notificacioncorreo/NotificacionCorreoEnviadaListener.java

```
/**
 * Caso de uso 3: marca el pedido con emailEnviado = true cuando msvc-envio confirma que salió el correo.
 *
 * Errores:
 *   - Recuperables: el pedido todavía no está guardado (el aviso llegó antes del fin del checkout) o la
     base de
 *     datos falla temporalmente. Se reintentan con espera.
 *   - No recuperables: evento sin pedidoId. Van directo a la DLQ.
 * Es idempotente: marcar dos veces el mismo pedido deja el mismo resultado.
 */
@Component
public class NotificacionCorreoEnviadaListener extends
    ConsumidorConReintentosYDeadLetterQueue<NotificacionCorreoEnviadaEvent> {

    private final PedidoRepository pedidoRepository;

    public NotificacionCorreoEnviadaListener(
            PedidoRepository pedidoRepository,
            ReintentoDeMensajesPublisher reintentoPublisher,
            @Value("${rabbitmq.queue.correo-enviado-reintento}") String reintentoQueueName,
            @Value("${rabbitmq.reintento.maximo-reintentos}") int maximoReintentos,
            @Value("${rabbitmq.reintento.header-numero-reintento}") String headerNumeroReintento
    ) {
        super(reintentoPublisher, reintentoQueueName, maximoReintentos, headerNumeroReintento);
```

```
        this.pedidoRepository = pedidoRepository;
    }

    @RabbitListener(queues = "${rabbitmq.queue.correo-enviado}")
    public void onCorreoEnviado(
            NotificacionCorreoEnviadaEvent event,
            Channel channel,
            @Header(AmqpHeaders.DELIVERY_TAG) long deliveryTag,
            @Headers Map<String, Object> headers
    ) throws IOException {
        procesarConConfirmacionManual(event, channel, deliveryTag, headers);
    }
    @Override
    protected void procesar(NotificacionCorreoEnviadaEvent event) {
        if (event == null || event.pedidoId() == null) {
            throw new IllegalArgumentException("El aviso de correo enviado no trae pedidoId");
        }

        Pedido pedido = pedidoRepository.findById(event.pedidoId())
                .orElseThrow(() -> new PedidoTodaviaNoRegistradoException(event.pedidoId()));

        if (pedido.isEmailEnviado()) {
            log.info("El pedido {} ya estaba marcado con el correo enviado; mensaje duplicado",
                event.numeroPedido());
            return;
        }
        pedido.setEmailEnviado(true);
        pedidoRepository.save(pedido);
    }

    @Override
    protected boolean esErrorRecuperable(Exception e) {
        return e instanceof PedidoTodaviaNoRegistradoException
                || e instanceof TransientDataAccessException
                || e instanceof DataAccessResourceFailureException
                || e instanceof CannotCreateTransactionException;
    }

    @Override
    protected String describir(NotificacionCorreoEnviadaEvent event) {
        return event.numeroPedido();
    }
}
```

El aviso podría llegar antes de que termine la transacción del checkout. Por eso "pedido no encontrado" se trata como error **recuperable**: se reintenta a los 10 segundos, cuando el pedido ya está guardado.

10.8 Los consumidores de las DLQ

msvc-envio · listener/RegistroDeMensajesNoEntregados.java

```
/**
 * Registra en logs un mensaje leído desde una DLQ (cola de origen, motivo, headers y contenido) y lo
     confirma.
 * Lo usan los consumidores de DLQ de cada caso de uso.
 */
@Component
public class RegistroDeMensajesNoEntregados {

    public void registrarYConfirmar(Logger log, Message message, Channel channel) throws IOException {
        MessageProperties properties = message.getMessageProperties();
        String cuerpo = new String(message.getBody(), StandardCharsets.UTF_8);
```

```
        // RabbitMQ agrega el header x-death con el historial; la primera entrada es la más reciente.
        List<Map<String, ?>> historialDeMuertes = properties.getXDeathHeader();
        Object colaOrigen = "desconocida";
        Object motivo = "desconocido";
        if (historialDeMuertes != null && !historialDeMuertes.isEmpty()) {
            Map<String, ?> ultimaMuerte = historialDeMuertes.get(0);
            colaOrigen = ultimaMuerte.get("queue");
            motivo = ultimaMuerte.get("reason");
        }

        log.error("Mensaje no entregado registrado desde la DLQ. colaOrigen={}, motivo={}, headers={},
            cuerpo={}",
                colaOrigen, motivo, properties.getHeaders(), cuerpo);

        // ACK: el mensaje ya quedó registrado en logs y se retira de la DLQ
        channel.basicAck(properties.getDeliveryTag(), false);
    }
}
```

Cada DLQ tiene su propio consumidor dentro de su dominio (por ejemplo *RegistroEnvioDeadLetterQueueListener*), que usa el conversor sin transformación y delega el registro en este componente.

> Los consumidores de DLQ hacen ACK después de registrar el mensaje, así que las DLQ quedan vacías y los mensajes muertos se revisan en los logs. Si se prefiere conservarlos en el broker para reprocesarlos, habría que no consumir las DLQ o reenviarlos a una cola de estacionamiento.

10.9 msvc-rabbitmq-admin: la API de administración

Este microservicio expone una API REST para crear y eliminar colas, exchanges y bindings. La lógica está encapsulada en *RabbitAdminService*, que usa *RabbitAdmin*; el controlador solo valida la entrada y traduce a HTTP.

Método y ruta	Entrada	Respuesta correcta	Errores
POST /api/admin/queues	{"name": "mi.cola"}	201 + mensaje JSON	400 si el nombre es vacío, inválido o mayor a 255; 409 si RabbitMQ la rechaza (por ejemplo, ya existe con otros argumentos).
DELETE /api/admin/queues/{name}	Nombre en la ruta	204 sin contenido	400 si el nombre es inválido.
POST /api/admin/exchanges	{"name": "mi.exchange"}	201 + mensaje JSON	400 / 409.
DELETE /api/admin/exchanges/{name}	Nombre en la ruta	204	400.
POST /api/admin/bindings	{"queueName", "exchangeName", "routingKey"}	201 + mensaje JSON	400 si falta o es inválido algún campo; 409 si la cola o el exchange no existen.
DELETE /api/admin/bindings	?queueName=&exchangeName=&routingKey=	204	400 si falta un parámetro o es inválido.

msvc-rabbitmq-admin · service/RabbitAdminService.java

```
@Service
public class RabbitAdminService {

    private final RabbitAdmin rabbitAdmin;

    public RabbitAdminService(RabbitAdmin rabbitAdmin) {
        this.rabbitAdmin = rabbitAdmin;
    }

    public void createQueue(String name) {
        Queue queue = new Queue(name, true);
        rabbitAdmin.declareQueue(queue);
    }

    public void deleteQueue(String name) {
        rabbitAdmin.deleteQueue(name);
    }

    public void createExchange(String name) {
        DirectExchange exchange = new DirectExchange(name, true, false);
        rabbitAdmin.declareExchange(exchange);
    }

    public void deleteExchange(String name) {
        rabbitAdmin.deleteExchange(name);
    }

    public void createBinding(String queueName, String exchangeName, String routingKey) {
        Queue queue = new Queue(queueName, true);
        DirectExchange exchange = new DirectExchange(exchangeName, true, false);
        Binding binding = BindingBuilder.bind(queue).to(exchange).with(routingKey);
        rabbitAdmin.declareBinding(binding);
    }

    public void deleteBinding(String queueName, String exchangeName, String routingKey) {
        Binding binding = new Binding(queueName, Binding.DestinationType.QUEUE, exchangeName, routingKey,
            null);
        rabbitAdmin.removeBinding(binding);
    }
}
```

La validación usa anotaciones de Jakarta Validation en los DTO (*@NotBlank*, *@Pattern*, *@Size*) y en los parámetros de ruta; las reglas comunes viven en *RabbitResourceNameRules*. *RabbitAdminApiExceptionHandler* convierte cada error en un JSON *{"error", "details"}*:

Respuestas reales obtenidas en la prueba local

```
POST /api/admin/queues  {"name": ""}
HTTP 400
{"error":"Datos de entrada inválidos",
 "details":["name: El nombre contiene caracteres inválidos","name: El nombre no puede estar vacío"]}

POST /api/admin/bindings  {"queueName":"no.existe","exchangeName":"prueba.exchange","routingKey":"k"}
HTTP 409
{"error":"RabbitMQ rechazó la operación",
 "details":["... reply-code=404, reply-text=NOT_FOUND - no queue 'no.existe' in vhost '/' ..."]}
```

11. Qué pasa en cada caso

Esta sección recorre cada escenario de punta a punta. Todos se probaron contra el stack local de Docker con la versión actual del código, salvo el envío real por SMTP (caso 2), que se cubre con pruebas unitarias porque en local SMTP está deshabilitado. Los logs mostrados son los reales, recortados para que quepan.

Caso 1: compra exitosa

1. El cliente confirma el pago; msvc-pago valida stock y carrito por HTTP y guarda el pedido.
2. PagoEventPublisher publica *pago.completado* y espera la confirmación de RabbitMQ.
3. El exchange direct entrega una copia a *envio.queue* y otra a *notificacion.correo.queue*; el checkout ya respondió al cliente.
4. Los dos consumidores trabajan en paralelo: uno registra el envío; el otro envía el correo (o, sin SMTP, registra la notificación como omitida).
5. Cada uno hace su propio **ACK**.

Logs

```
PagoEventPublisher            : Publicando evento pago.completado para pedido: PED-97510599
PagoEventPublisher            : Evento confirmado por RabbitMQ para pedido: PED-97510599
...RegistroEnvioListener      : Mensaje recibido para pedido: PED-97510599 (reintentos realizados: 0)
...NotificacionCorreoListener : Mensaje recibido para pedido: PED-97510599 (reintentos realizados: 0)
...RegistroEnvioListener      : ACK enviado para pedido: PED-97510599
...NotificacionCorreoListener : SMTP deshabilitado: se omite el correo del pedido PED-97510599
...NotificacionCorreoListener : ACK enviado para pedido: PED-97510599
```

> Resultado: un solo evento alimenta dos tareas independientes; el cliente no espera ninguna.

Caso 2: correo enviado y aviso a msvc-pago

1. Con SMTP habilitado, PagoCompletadoNotificacionCorreoListener envía el comprobante y guarda la notificación como ENVIADA.
2. Publica *notificacion.correo.enviada* en *pedidos.exchange*.
3. NotificacionCorreoEnviadaListener de msvc-pago marca el pedido con *emailEnviado* = *true* y hace ACK.
4. En la prueba local (sin SMTP) el aviso se publicó a mano en RabbitMQ; el envío del correo y la publicación del aviso se comprueban con PagoCompletadoNotificacionCorreoListenerTest.

Logs

```
.p.l.n.NotificacionCorreoEnviadaListener : Mensaje recibido para pedido: PED-97510599 (reintentos
    realizados: 0)
.p.l.n.NotificacionCorreoEnviadaListener : ACK enviado para pedido: PED-97510599

pago_db> select numero_pedido, email_enviado from pedidos where numero_pedido = 'PED-97510599';
PED-97510599 | t
```

> Resultado: el pedido refleja que el correo salió, sin que msvc-pago llame a msvc-envio.

Caso 3: mensajes duplicados

1. Llega por segunda vez el mismo *pago.completado* (o el mismo aviso de correo enviado).
2. Cada consumidor detecta que ya lo procesó: existe el envío, la notificación o la marca del pedido.
3. Hace ACK sin repetir el efecto.

Logs

```
...RegistroEnvioListener      : El pedido PED-97510599 ya tiene un envío registrado; mensaje duplicado
...NotificacionCorreoListener : El pedido PED-97510599 ya fue notificado; mensaje duplicado
NotificacionCorreoEnviadaListener : El pedido PED-97510599 ya estaba marcado con el correo enviado; mensaje
    duplicado
```

Resultado: consumidores idempotentes; la entrega at-least-once no duplica envíos ni correos.

Caso 4: evento con datos incompletos (error no recuperable)

1. Llega un *pago.completado* sin pedidoId, usuarioId, correo o dirección.
2. Cada consumidor lo valida y lanza *IllegalArgumentException*: no es un error recuperable.
3. NACK sin requeue: RabbitMQ lo reenvía a la DLQ de cada caso de uso con motivo *rejected*.
4. Los consumidores de las DLQ lo registran en logs con cola de origen, motivo, headers y cuerpo.

Logs

```
...RegistroEnvioListener      : Error no recuperable procesando pedido PRUEBA-INCOMPLETO tras 0 reintentos.
                                Enviando a DLQ: El evento no trae pedidoId o usuarioId
...NotificacionCorreoListener : Error no recuperable procesando pedido PRUEBA-INCOMPLETO tras 0 reintentos.
                                Enviando a DLQ: El evento no trae pedidoId, numeroPedido o email
RegistroEnvioDeadLetterQueueListener      : Mensaje no entregado registrado desde la DLQ.
    colaOrigen=envio.queue,
                                            motivo=rejected, ...
NotificacionCorreoDeadLetterQueueListener : Mensaje no entregado registrado desde la DLQ.
                                            colaOrigen=notificacion.correo.queue, motivo=rejected, ...
```

Resultado: no se gastan reintentos en un mensaje que nunca podrá procesarse; queda trazado en logs.

Caso 5: JSON inválido

1. Llega un mensaje cuyo cuerpo no es JSON válido.
2. El conversor JSON falla antes de llegar al método; Spring lo considera un error **fatal** (*ConditionalRejectingErrorHandler*) y lo rechaza sin requeue.
3. El mensaje va a cada DLQ. Sus consumidores usan el conversor sin transformación, así que pueden leerlo y registrarlo aunque el JSON esté roto.

Logs

```
SimpleMessageListenerContainer : ListenerExecutionFailedException: Failed to convert message
RegistroEnvioDeadLetterQueueListener      : Mensaje no entregado registrado desde la DLQ.
    colaOrigen=envio.queue,
                                            motivo=rejected, ... cuerpo={esto no es json
NotificacionCorreoDeadLetterQueueListener : Mensaje no entregado registrado desde la DLQ.
                                            colaOrigen=notificacion.correo.queue, motivo=rejected, ...
```

Resultado: un mensaje corrupto no bloquea las colas ni desaparece sin dejar rastro.

Caso 6: la base de datos falla (error recuperable) y se agotan los reintentos

1. Llega un evento válido mientras la base de msvc-envio no responde.
2. Al consultar o guardar, JPA no obtiene conexión: error recuperable.
3. Cada consumidor publica una copia en su cola de reintento con *x-numero-reintento* incrementado y hace ACK del original; a los 10 s la copia vuelve a la cola principal.
4. Si la base vuelve antes del tercer reintento, el siguiente intento termina con ACK. Si no, al cuarto intento el mensaje va a la DLQ.

Logs

```
...RegistroEnvioListener : Error recuperable procesando pedido PRUEBA-BD-CAIDA. Programado reintento 1 de 3:
                           Unable to acquire JDBC Connection [HikariPool-1 - Connection is not available
                               ...]
...RegistroEnvioListener : Mensaje recibido para pedido: PRUEBA-BD-CAIDA (reintentos realizados: 1)
...RegistroEnvioListener : Error recuperable procesando pedido PRUEBA-BD-CAIDA. Programado reintento 2 de
    3: ...
...RegistroEnvioListener : Error recuperable procesando pedido PRUEBA-BD-CAIDA. Programado reintento 3 de
    3: ...
...RegistroEnvioListener : Mensaje recibido para pedido: PRUEBA-BD-CAIDA (reintentos realizados: 3)
...RegistroEnvioListener : Error recuperable procesando pedido PRUEBA-BD-CAIDA tras 3 reintentos. Enviando
    a DLQ: ...
RegistroEnvioDeadLetterQueueListener : Mensaje no entregado registrado desde la DLQ. colaOrigen=envio.queue,
                                       motivo=rejected, headers={x-numero-reintento=3, ...}
(la notificación por correo muestra la misma secuencia, en paralelo y con su propia cola de reintento)
```

Resultado: una caída temporal no pierde mensajes, y los reintentos tienen límite: no hay bucles infinitos.

Cada intento con la base caída tarda unos 28 segundos, porque es lo que espera el pool de conexiones antes de fallar. Más los 10 s de espera, agotar los 3 reintentos toma algo más de 2 minutos.

Caso 7: el aviso de correo llega antes que el pedido

1. msvc-pago recibe *notificacion.correo.enviada* para un pedido que aún no encuentra en su base.
2. Lo trata como error recuperable (*PedidoTodaviaNoRegistradoException*) y lo reintenta a los 10 s.
3. Si tras 3 reintentos el pedido sigue sin existir, el aviso va a *pago.correo-enviado.dlq*.

Logs

```
NotificacionCorreoEnviadaListener : Error recuperable procesando pedido PED-INEXISTENTE. Programado
    reintento 1 de 3:
                                    El pedido 00000000-0000-0000-0000-000000000000 todavía no está
                                        registrado
NotificacionCorreoEnviadaListener : Mensaje recibido para pedido: PED-INEXISTENTE (reintentos realizados: 1)
NotificacionCorreoEnviadaListener : Error recuperable procesando pedido PED-INEXISTENTE. Programado
    reintento 2 de 3: ...
```

Resultado: una carrera de tiempos entre servicios se resuelve sola con el reintento.

Caso 8: msvc-envio está detenido

Los mensajes se acumulan en *envio.queue* y *notificacion.correo.queue* (durables) mientras no haya consumidor. Cuando msvc-envio vuelve, los procesa. Los clientes siguen comprando sin notarlo. **Este es el principal beneficio del desacoplamiento.**

Caso 9: RabbitMQ no confirma la publicación

Si RabbitMQ está caído, rechaza el mensaje, no responde en 5 s o el mensaje no llega a ninguna cola, *PagoEventPublisher* lanza *PublicacionDeEventoFallidaException* (cubierto por *PagoEventPublisherTest*). PagoController la captura y registra en logs el evento completo para reprocesarlo; la compra sigue siendo válida. El reproceso es manual: automatizarlo requiere el patrón Outbox (sección 13).

Caso 10: administración por API

Crear una cola, un exchange y un binding responde 201; eliminarlos, 204. Un nombre vacío o con espacios responde 400 con el detalle del error; un binding hacia una cola inexistente o redeclarar *envio.queue* con otros argumentos responde 409 con el motivo que entrega RabbitMQ.

12. Cómo comprobar que funciona

12.1 En local (Docker Desktop)

1. Levantar el stack: *docker compose up -d --build* en *pedidos-backend*.
2. Abrir la consola de RabbitMQ en *http://localhost:15672* (guest/guest). En **Exchanges** debe aparecer *pedidos.exchange*; en **Queues**, nueve colas: las tres principales y las tres DLQ con 1 consumidor, y las tres *.reintento.queue* con 0.
3. Hacer una compra en el frontend y revisar los logs (caso 1).
4. Para el caso 2, publicar un mensaje desde la consola: en *pedidos.exchange* → **Publish message**, routing key *pago.completado*, propiedad *content_type=application/json* y cuerpo *{"numeroPedido":"PRUEBA-DLQ"}*.
5. Para los casos 4 y 5: *docker compose stop db-envio*, hacer una compra y observar los reintentos; luego *docker compose start db-envio*.

Seguir solo los logs de mensajería

```
docker compose logs -f msvc-envio msvc-pago | grep -E "Listener|PagoEventPublisher"
```

12.2 En AWS (EC2 backend, por Session Manager)

Comandos de verificación en la EC2 backend

```
# Colas, mensajes pendientes y consumidores
sudo docker compose -f docker-compose.prod.yml exec rabbitmq \
  rabbitmqctl list_queues name messages consumers

# Logs de mensajería de los últimos 5 minutos
sudo docker compose -f docker-compose.prod.yml logs --since 5m msvc-envio msvc-pago \
  | grep -E "Listener|PagoEventPublisher"

# Publicar un evento incompleto para probar la DLQ (la red se ve con: sudo docker network ls)
sudo docker run --rm --network pedidos360_default curlimages/curl:8.10.1 -s -u guest:guest \
  -H 'Content-Type: application/json' \
  -X POST http://rabbitmq:15672/api/exchanges/%2F/pedidos.exchange/publish \
  -d '{"properties":{"content_type":"application/json"},"routing_key":"pago.completado",
       "payload":"{\"numeroPedido\":\"PRUEBA-DLQ\"}","payload_encoding":"string"}'

# API de administración
curl -s -X POST localhost:8090/api/admin/queues -H 'Content-Type: application/json' -d
    '{"name":"demo.cola"}'
curl -s -X POST localhost:8090/api/admin/queues -H 'Content-Type: application/json' -d '{"name":""}'
curl -s -o /dev/null -w '%{http_code}\n' -X DELETE localhost:8090/api/admin/queues/demo.cola
```

> En AWS la base de datos es RDS, que no conviene apagar para probar reintentos. Los casos de base de datos caída se demuestran mejor en local.

12.3 Pruebas automáticas

Cada servicio con mensajería tiene pruebas unitarias con JUnit 5 y Mockito, que no necesitan RabbitMQ ni base de datos: se ejecutan con *./mvnw test* en la carpeta del servicio.

Servicio	Clase de prueba	Qué comprueba
msvc-envio	PagoCompletadoRegistroEnvioListenerTest (6)	ACK al guardar; NACK a DLQ con datos inválidos; duplicado sin segundo envío; reintento con base caída; DLQ con reintentos agotados o sin poder reintentar.
msvc-envio	PagoCompletadoNotificacionCorreoListenerTest (6)	Notificación omitida sin SMTP; correo enviado y aviso a pago; reintento con SMTP caído; DLQ con credenciales rechazadas; sin reenvío de duplicados; ACK aunque falle el aviso.
msvc-pago	NotificacionCorreoEnviadaListenerTest (4)	Marca del pedido; reintento si el pedido aún no existe; DLQ tras los reintentos o sin pedidoId.
msvc-pago	PagoEventPublisherTest (4)	Confirmación aceptada, rechazada, sin cola destino y sin respuesta.
msvc-rabbitmq-admin	RabbitAdminControllerTest (7) y RabbitAdminServiceTest (4)	Códigos 201, 204, 400 y 409 con sus mensajes; declaración y eliminación de colas, exchanges y bindings.

13. Evaluación del diseño y mejoras propuestas

13.1 Lo que está bien resuelto

- Comunicación por **eventos** con transferencia de estado: los consumidores no llaman de vuelta al productor.
- Tres casos de uso con sus propios beans de Queue, Exchange y Binding, y nombres centralizados en *application.yml*.
- Publish/subscribe real: dos consumidores independientes para el mismo evento.
- Configuración separada del negocio: clases *@Configuration* por caso de uso, publicadores, servicios y consumidores agrupados por dominio.
- ACK manual con rutas de error diferenciadas y documentadas, iguales en todos los consumidores.
- Consumidores idempotentes y confirmaciones del publicador.
- Microservicio de administración con API REST, servicio dedicado y validación con mensajes claros.
- Pruebas unitarias de consumidores, publicador y administrador.

13.2 Mejoras pendientes

N°	Mejora	Por qué
1	Patrón Transactional Outbox en msvc-pago.	Hoy, si RabbitMQ no confirma, el evento queda en logs y su reproceso es manual. Con un outbox se publicaría automáticamente cuando el broker vuelva.
2	Eliminar el endpoint síncrono *POST /internal/envios/confirmacion* de msvc-envio.	Ya nadie lo llama; dejarlo puede confundir sobre cómo se comunican los servicios.

N°	Mejora	Por qué
3	Proteger msvc-rabbitmq-admin: exponerlo solo a través del BFF con rol Administrador.	Hoy cualquiera con acceso a la red interna puede borrar colas.
4	Exchange propio para las DLQ, backoff exponencial y colas quorum.	Topología más explícita, mejor recuperación y alta disponibilidad en producción.

14. Resumen: por qué Pedidos360 necesita una cola de mensajes

1. **Para no hacer esperar al cliente**: el checkout responde apenas se confirma el pago; registrar el envío y mandar el correo ocurre en segundo plano.
2. **Para que una falla no tumbe la compra**: si msvc-envio, su base de datos o el servidor de correo fallan, el pago igual se confirma y el trabajo pendiente espera en la cola.
3. **Para desacoplar servicios**: msvc-pago anuncia un hecho y no conoce a quién le interesa. Agregar un nuevo interesado (boleta, puntos, analítica) no requiere tocar msvc-pago.
4. **Para manejar errores con control**: los fallos temporales se reintentan con espera y límite; los permanentes se apartan a la DLQ y quedan registrados en vez de perderse.
5. **Para escalar**: ante más carga, basta con levantar más réplicas del consumidor; la cola reparte el trabajo.

Es exactamente lo que pide el enunciado de la evaluación: incorporar colas, exchanges y DLQs para gestionar tareas asíncronas como notificaciones, tickets o documentos, **sin afectar la lógica existente del sistema**.

15. Glosario

Término	Definición
ACK / NACK	Confirmación positiva o negativa del consumidor sobre un mensaje.
AMQP	Advanced Message Queuing Protocol, protocolo abierto que implementa RabbitMQ (versión 0-9-1).
Binding	Regla que une un exchange con una cola, normalmente mediante una routing key.
Broker	Servidor intermediario que recibe, almacena y entrega mensajes.
Consistencia eventual	Los datos de distintos servicios coinciden después de un tiempo, no al instante.
DLX / DLQ	Dead Letter Exchange / Queue: exchange y cola a donde van los mensajes que no se pudieron procesar.
Durable	Recurso que sobrevive a un reinicio del broker.
Exchange	Componente que recibe los mensajes del productor y los enruta a las colas.
Idempotencia	Procesar el mismo mensaje varias veces produce el mismo resultado que procesarlo una vez.
Prefetch	Número máximo de mensajes sin confirmar que un consumidor puede tener.
Requeue	Devolver un mensaje rechazado a su cola original.
Routing key	Etiqueta del mensaje que el exchange compara con los bindings.
TTL	Time To Live: tiempo de vida de un mensaje antes de expirar.
x-death	Header que RabbitMQ agrega con la historia de dead-lettering de un mensaje.

Referencias

- Documentación oficial de RabbitMQ: *rabbitmq.com/docs* (AMQP 0-9-1 Model, Confirms, Dead Letter Exchanges, TTL, Quorum Queues).
- Documentación de Spring AMQP: *docs.spring.io/spring-amqp/reference*.
- Gregor Hohpe y Bobby Woolf, *Enterprise Integration Patterns*, Addison-Wesley, 2003.
- Chris Richardson, *Microservices Patterns*, Manning, 2018 (Saga, Transactional Outbox, Idempotent Consumer).
- Código fuente de Pedidos360: *pedidos-backend/msvc-pago*, *msvc-envio* y *msvc-rabbitmq-admin*.